

```
pip install scikit-learn scipy seaborn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (1.16.3)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.53.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.4)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

print("Train shape:", x_train.shape)
print("Test shape:", x_test.shape)
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 19s 0us/step
Train shape: (50000, 32, 32, 3)
Test shape: (10000, 32, 32, 3)
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

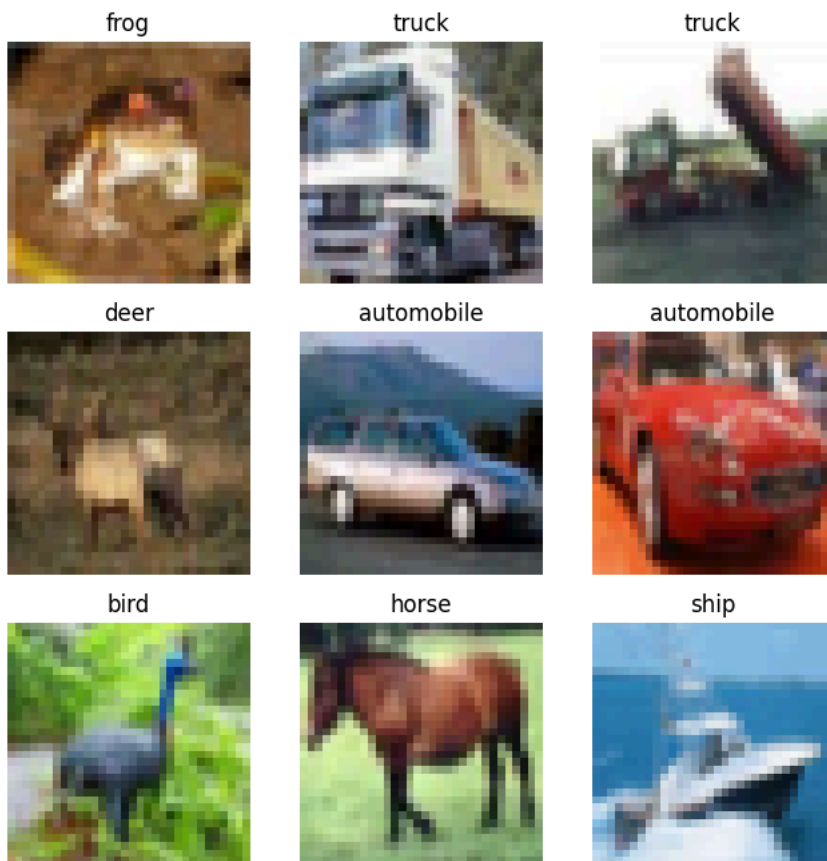
print("Train shape:", x_train.shape)
print("Test shape:", x_test.shape)
```

```
Train shape: (50000, 32, 32, 3)
Test shape: (10000, 32, 32, 3)
```

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
```

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')
plt.show()
```



```
def create_model(learning_rate=0.001, dropout_rate=0.5):
    model = models.Sequential()

    # Conv Block 1
    model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)))
    model.add(layers.MaxPooling2D(2,2))

    # Conv Block 2
    model.add(layers.Conv2D(64, (3,3), activation='relu'))
    model.add(layers.MaxPooling2D(2,2))

    # Conv Block 3
    model.add(layers.Conv2D(128, (3,3), activation='relu'))
    model.add(layers.MaxPooling2D(2,2))

    # Dense
    model.add(layers.Flatten())
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dropout(dropout_rate))
    model.add(layers.Dense(10, activation='softmax'))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```
configs = [
    {"lr":0.001, "dropout":0.5},
    {"lr":0.0001, "dropout":0.3},
    {"lr":0.01, "dropout":0.6}
]

histories = []

for cfg in configs:
    print(f"\nTraining with LR={cfg['lr']} Dropout={cfg['dropout']}")

    model = create_model(cfg['lr'], cfg['dropout'])

    history = model.fit(
        x_train, y_train,
        epochs=10,
```

```
validation_data=(x_test, y_test),
verbose=1
)
```

```
histories.append((cfg, history, model))
```

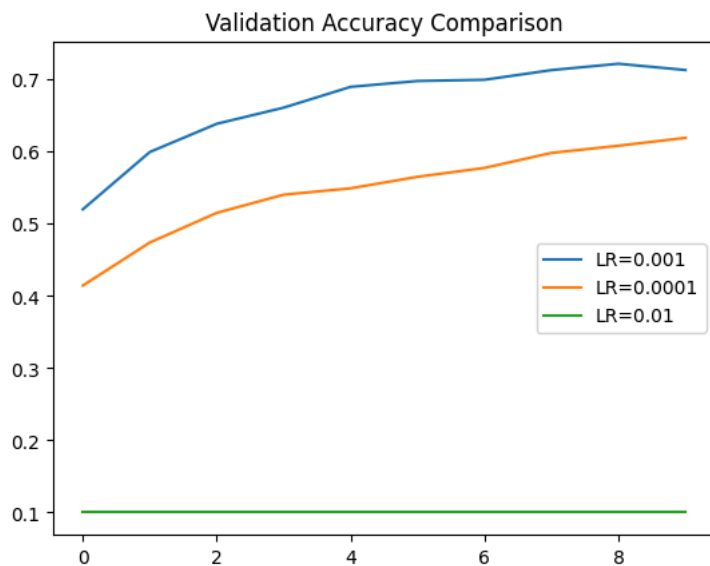
```
Epoch 4/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.6512 - loss: 0.9914 - val_accuracy: 0.6610 - val_loss: 0.9575
Epoch 5/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.6790 - loss: 0.9191 - val_accuracy: 0.6898 - val_loss: 0.8756
Epoch 6/10
1563/1563 ————— 8s 5ms/step - accuracy: 0.7014 - loss: 0.8578 - val_accuracy: 0.6979 - val_loss: 0.8568
Epoch 7/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.7176 - loss: 0.8081 - val_accuracy: 0.6996 - val_loss: 0.8680
Epoch 8/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.7313 - loss: 0.7657 - val_accuracy: 0.7130 - val_loss: 0.8436
Epoch 9/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.7458 - loss: 0.7266 - val_accuracy: 0.7218 - val_loss: 0.8252
Epoch 10/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.7558 - loss: 0.6965 - val_accuracy: 0.7131 - val_loss: 0.8495

Training with LR=0.0001 Dropout=0.3
Epoch 1/10
1563/1563 ————— 17s 7ms/step - accuracy: 0.2978 - loss: 1.8859 - val_accuracy: 0.4145 - val_loss: 1.6311
Epoch 2/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.4189 - loss: 1.5857 - val_accuracy: 0.4741 - val_loss: 1.4611
Epoch 3/10
1563/1563 ————— 8s 5ms/step - accuracy: 0.4723 - loss: 1.4606 - val_accuracy: 0.5152 - val_loss: 1.3504
Epoch 4/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.5060 - loss: 1.3724 - val_accuracy: 0.5404 - val_loss: 1.2988
Epoch 5/10
1563/1563 ————— 8s 5ms/step - accuracy: 0.5275 - loss: 1.3171 - val_accuracy: 0.5492 - val_loss: 1.2705
Epoch 6/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.5477 - loss: 1.2679 - val_accuracy: 0.5652 - val_loss: 1.2173
Epoch 7/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.5654 - loss: 1.2296 - val_accuracy: 0.5775 - val_loss: 1.1910
Epoch 8/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.5793 - loss: 1.1897 - val_accuracy: 0.5983 - val_loss: 1.1381
Epoch 9/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.5941 - loss: 1.1523 - val_accuracy: 0.6082 - val_loss: 1.1076
Epoch 10/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.6035 - loss: 1.1237 - val_accuracy: 0.6191 - val_loss: 1.0918

Training with LR=0.01 Dropout=0.6
Epoch 1/10
1563/1563 ————— 14s 7ms/step - accuracy: 0.1017 - loss: 2.3044 - val_accuracy: 0.1000 - val_loss: 2.3040
Epoch 2/10
1563/1563 ————— 8s 5ms/step - accuracy: 0.0979 - loss: 2.3039 - val_accuracy: 0.1000 - val_loss: 2.3049
Epoch 3/10
1563/1563 ————— 7s 4ms/step - accuracy: 0.0999 - loss: 2.3040 - val_accuracy: 0.1000 - val_loss: 2.3038
Epoch 4/10
1563/1563 ————— 8s 5ms/step - accuracy: 0.1024 - loss: 2.3037 - val_accuracy: 0.1000 - val_loss: 2.3041
Epoch 5/10
1563/1563 ————— 10s 5ms/step - accuracy: 0.0997 - loss: 2.3041 - val_accuracy: 0.1000 - val_loss: 2.3046
Epoch 6/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.0995 - loss: 2.3041 - val_accuracy: 0.1000 - val_loss: 2.3040
Epoch 7/10
1563/1563 ————— 10s 4ms/step - accuracy: 0.0989 - loss: 2.3039 - val_accuracy: 0.1000 - val_loss: 2.3059
Epoch 8/10
1563/1563 ————— 8s 5ms/step - accuracy: 0.1005 - loss: 2.3040 - val_accuracy: 0.1000 - val_loss: 2.3039
Epoch 9/10
1563/1563 ————— 7s 5ms/step - accuracy: 0.0980 - loss: 2.3039 - val_accuracy: 0.1000 - val_loss: 2.3040
Epoch 10/10
1563/1563 ————— 8s 5ms/step - accuracy: 0.0982 - loss: 2.3041 - val_accuracy: 0.1000 - val_loss: 2.3031
```

```
for cfg, history, _ in histories:
    plt.plot(history.history['val_accuracy'], label=f"LR={cfg['lr']}")

plt.title("Validation Accuracy Comparison")
plt.legend()
plt.show()
```



```
best_model = histories[0][2] # choose best manually based on accuracy
```

```
test_loss, test_acc = best_model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.7131 - loss: 0.8495
Test Accuracy: 0.713100016117096
```

```
for i in range(5):
    pred = best_model.predict(x_test[i:i+1])
    print("Predicted:", class_names[np.argmax(pred)],
          "| Actual:", class_names[y_test[i][0]])
```

```
1/1 ————— 1s 567ms/step
Predicted: cat | Actual: cat
1/1 ————— 0s 48ms/step
Predicted: ship | Actual: ship
1/1 ————— 0s 39ms/step
Predicted: ship | Actual: ship
1/1 ————— 0s 38ms/step
Predicted: airplane | Actual: airplane
1/1 ————— 0s 38ms/step
Predicted: deer | Actual: frog
```

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

y_pred = best_model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = tf.math.confusion_matrix(y_test.flatten(), y_pred_classes)

plt.imshow(cm)
plt.title("Confusion Matrix")
plt.colorbar()
plt.show()
```

313/313 1s 3ms/step

Confusion Matrix

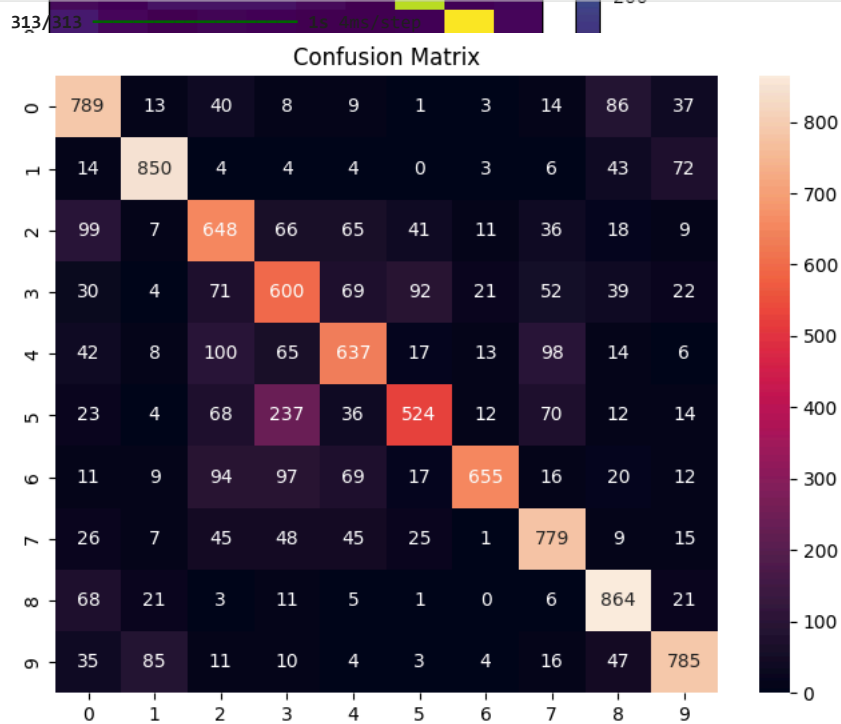


```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
y_pred = best_model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
```

```
cm = confusion_matrix(y_test, y_pred_classes)
```

```
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```



```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

|   | precision | recall | f1-score | support |
|---|-----------|--------|----------|---------|
| 0 | 0.69      | 0.79   | 0.74     | 1000    |
| 1 | 0.84      | 0.85   | 0.85     | 1000    |
| 2 | 0.60      | 0.65   | 0.62     | 1000    |
| 3 | 0.52      | 0.60   | 0.56     | 1000    |
| 4 | 0.68      | 0.64   | 0.66     | 1000    |